Lab 2b (Lower limb movement detection, EMG collection) – data processing

- **For questions about this lab please contact the TAs**
- **Files to be uploaded on Moodle:**
  1. **Report and collected data. The report should include the names, ETH IDs, and email addresses of all students. It should be submitted as either a PDF or Word document, named "groupXX_lab2". The report should include the answers to all questions in the lab instructions (Lab2a+Lab2b).**
  2. **Python (.py) or jupyter notebook (.ipynb) files named "groupXX_lab2" with XX being the group number.**
- **Achieving a high classification accuracy is not a necessary requirement for this assignment.**
- **Please read the whole document before starting the coding assignment. This will help you better understand the assignment, and better design your code for reusability / generalizability.**

Instructions

1. Load and organize the data for each activity into a pandas DataFrame, with 4 columns: three for signals (EMG quad, EMG calf, strain) and one for labels (manually assigned based on activity). Use the following labels:
   - Stand = 0
   - Squat = 1
   - Toe raise = 2
   - Kick back=3
   - Knee raise = 4

1 EMG_quad	2 EMG_calf	3 strain_sensor	4 activity
484	491	817	0
498	490	817	0
494	491	817	0
489	490	817	0
490	491	817	0
476	491	817	0
489	490	817	0

2. Discard segments without activity (e.g., NaN values at start/end). **Note:** use df.dropna(). (https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.dropna.html).
3. Clean the signals by removing outliers (any spikes) from EMG signals and low pass filter the signal from the strain sensor. For example, a low pass filter with a 10Hz cutoff can be used. Assume a sampling frequency of 100 Hz.

   Note: For filtering, you can implement a simple low pass Butterworth filter using the scipy library. Use a zero-phase digital filtering method. (e.g. from scipy.signal import butter, filtfilt).
   Outlier removal can be performed by setting a threshold manually or with the provided python function (remove_outliers(), **see appendix**).

   Make sure to observe the data before and after these operations to check that you obtained the desired result and to observe the size of the dataset before and after (e.g. for outlier removal). If the ranges for output of the two EMG sensors are very different, it may be best to apply outlier removal separately to the two EMG sensors data, e.g. first on calf EMG, then on quad EMG.

4. Split the pandas activity DataFrame into training and test data with a 80%, 20% ratio, keeping the signal columns and the label column together in each DataFrame.
   Hint:
```
from sklearn.model_selection import train_test_split
train_df, test_df = train_test_split(activity_df, train_size=0.8,
random_state=42, shuffle=False)
```

5. Extract features in a moving window approach, with a window size ws=100 and each window separated by 1 point (99% window overlap). Start at the sample = window size, i.e. discard samples before that.
   Extract the following **features from the EMG signals**:
   - Root-Mean-Squares (RMS): implement (do not use python built-in function)
   - Waveform length: implement (do not use python built-in function)
   - Fourth order autoregressive coefficients (AR): use the function aryule **provided in the appendix** (Yule Walker method).

   Extract the **RMS from the strain signal.**
   Note: Implement a separate function for each feature, e.g., one function to calculate RMS, one function to calculate waveform length and one function to calculate the AR coefficients.

   Hint: You should have the following features per window: RMS, WL, 4 AR coeffs, per EMG channel (2 channels = 12 features) + RMS for strain (1 feature) = 13 total**. Assign labels to each window. See appendix for pseudo-code.**

Variable	Value
n_strain_channels	1 (one sensor on the knee)
n_strain_features	1 (RMS)
n_emg_channels	2 (quad and calf)
n_emg_features	6 (1 RMS + 1 WL + 4 AR coeff)

0.4978	0.6186	0.9046	1.0365	-1.0151	-1.0163	-0.0048	0.0072	0.0024	0.0085	0.0393	0.0215	0.7319
0.4986	0.6180	0.9140	1.0503	-1.0212	-1.0228	0.0015	0.0120	0.0059	0.0164	0.0356	0.0157	0.7327
0.4991	0.6167	0.9253	1.0660	-1.0250	-1.0210	0.0090	0.0126	0.0112	0.0186	0.0265	0.0113	0.7334
0.4989	0.6148	0.9384	1.0743	-1.0236	-1.0076	0.0131	0.0075	0.0152	0.0124	0.0170	0.0094	0.7342

EMG Features | Strain Feature

6. Repeat points 1 to 5 for all the 5 activities, then combine all the data from the five activities together to have the following datasets:
   - train_features_all = dataset with features (training set) for all the 5 activities
   - train_labels_all = dataset with labels (training set) for all the 5 activities
   - test_features_all = dataset with features (test set) for all the 5 activities
   - test_labels_all = dataset with labels (test set) for all the 5 activities

   Hint: You can implement the pipeline (1 to 5) in a for loop
   The feature datasets— train_features_all, test_features_all —have 13 columns: 6 features (1 RMS, 1 wl, 4 AR coefficients) x 2 EMG channels + 1 feature for the strain channel.
   The label datasets—train_labels_all, test_labels_all —have 1 column (the label identifying the activity).

The plots in the questions to follow should represent the full datasets (all 5 activities), unless otherwise specified.

Question 2: Plot the following for the full dataset (all 5 activities):
   a. Cleaned data (with removed outliers) of the EMG signals.
   b. RMS of EMG signals.
   c. Waveform length of EMG signals.
   d. AR coefficients (4 per channel x 2 = 8; one subplot per channel or all in one).

Note: Include calf and quad data. Use subplots for structure. Concatenate all activities (along the time axis) into a single figure. See the appendix for an example (of a and b)

Question 3: Plot for the full dataset:
   a. Filtered signal from the strain sensor.
   b. RMS of strain signal.

Note: Concatenate all activities into a single figure.

Question 4: Implement PCA manually:

- Mean Subtraction (Centering)

- Calculate covariance matrix.
- Perform eigenvalue decomposition.
- Project data onto principal components (sort with np.argsort). Reduce to 2 dimensions for visualization.

Use the PCA method you implemented to reduce the data dimensions to 2D for cluster visualization purposes for the **dataset of two activities**: squat and toe raise.

Plot:

a. Scatterplot of the 2 main PCA components using the manually implemented PCA method, using the ground-truth labels as marker colors in the plot.
b. Analogous to a. for the testing dataset.
c. Are clusters from training similar to test set? Explain.
d. Compare and plot the result of your own implementation with the built-in function from sklearn (from sklearn.decomposition import PCA).

Note:See the appendix for an example of the scatterplot of the 2 main PCA components

Question 5: Use the k-mean clustering method from sk-learn to cluster **two activities**: squat and toe raise. **Note:** k-means should cluster the original 13-dimensional feature vectors.

a. Scatterplot of the 2 main PCA components on the whole dataset using the predicted/clustered labels as marker colors in the plot.
b. Compare the k-means clustered scatterplot with the ground truth scatterplot.

Note: See the appendix for an example of the scatterplot of the k-means clustered data.
Note: The numbers assigned to clusters during k-means clustering might be different from the ground-truth label (of activities), even when achieving perfect cluster delineation. In case the labels differ, manually swap the labels resulting from the k-means clustering (e.g., replace label of 0 with 1 and label of 1 with 0).
Use the pyplot scatter function for plotting the clusters plt.scatter()

Question 6: For the 2-class problem repeat the previous Question 2 and 5 for two other window sizes at your choice (ideally one that yields better and one worse accuracy, see **Question 7**). Plot the features for different window sizes. Which window size yields the best results? Why?
Note: You need to regenerate the features and labels datasets for each different window size.
This question is a replication of Question 2 and 5 for two more window sizes, i.e., for Question 6 you need to generate figures with the same structure you used for Question 2 and 5.

Question 7: For the 2 classes problem and using the built-in k-means function from sklearn, create a table (see a sample below) reporting the numerical values of the overall accuracy for the dataset that you obtained for the different window sizes.

Accuracy = (correct predictions / total samples) * 100.
Remap labels if needed (e.g., **swap 0/1 in case of match).**

	Window size 1	**Window size 2**	**Window size 3**
Dataset accuracy [%]			

Question 8: Comment on results:

a. How does the accuracy change with window size? Why? Refer to the plots of features for different window sizes.
b. What is the contribution of the signal from the strain sensor? Repeat the data processing without the strain sensor data for the window size yielding the highest accuracy and comment on your observations (e.g. how does accuracy change?)

Question 9: For best window size, use the k-means method to cluster the activities and visually inspect the resulting clusters after applying PCA for the 5 activities dataset. Plot the following:

a. Scatterplot of the 2 principal components for the 5-class dataset using the ground truth labels as colormarker
b. Analogous to a., but using the k-means labels as colormarker
c. Compare the results, comment on the resulting (visual) accuracy and explain potential causes.
d. How do the results change for separate runs of the k-means algorithm? Run the algorithm 4 more times with a **different seed** and make the scatterplot for each run. Do the results change? Why or why not?

Note: A direct comparison of the resulting labels is not trivial, as the number assigned during k-means for each cluster might be different than the ground-truth label, even when achieving perfect cluster delineation. **Computation of the accuracy or the use of the Hungarian algorithm mentioned below are not mandatory for this exercise.**
One could inspect the results by looking and the confusion matrix and assign the correct label as the arg max of each row, with the constraint that the label hasn't be assigned before.
Alternatively, use the Hungarian algorithm (also known as the Kuhn-Munkres algorithm) to find the best one-to-one correspondence between cluster labels and ground truth labels. This algorithm minimizes the total mismatch between the clusters and the ground truth classes.

Appendix

Remove outlier function

```
def remove_outliers(s, window, thresh=3):
# Calculate rolling mean and standard deviation over the specified window
    roll = s.rolling(window=window, min_periods=1, center=True)
    avg = roll.mean()  # Rolling mean
    std = roll.std(ddof=0)  # Rolling standard deviation (with N as
denominator)
    # Calculate z-scores by subtracting the rolling mean and dividing by
rolling std
    z = s.sub(avg).div(std)
    # Create a mask identifying values within the acceptable z-score
threshold
    m = z.between(-thresh, thresh)
    # Replace outliers (values outside threshold) with the rolling mean
    return s.where(m, avg)
# Apply the `remove_outliers` function to the first two columns (EMG) of the
DataFrame using a rolling window approach of size 50
df_outlier_removed = df.iloc[:, :2].apply(lambda x: remove_outliers(x,
window=50), axis=0)
```

Aryule function

```
def aryule(x, order):
    """
    Estimate AR model coefficients using Yule-Walker method
    Parameters:
    x (array-like): The time series data
    order (int): The order of the AR model

    Returns:
   a: The estimated AR coefficients
    """
    x = np.asarray(x)
```

```
    n = len(x)

    # Compute autocorrelation
    r = np.correlate(x, x, mode='full')[n-1:]
    r = r[:order+1] / n

    # Initialize
    a = np.zeros(order)
    e = np.zeros(order+1)
    k = np.zeros(order)

    e[0] = r[0]

    # Levinson-Durbin recursion
    for i in range(order):
        # Compute reflection coefficient
        k[i] = (r[i+1] - np.sum(a[:i] * r[i:0:-1])) / e[i]
        # Update AR coefficients
        a[i] = k[i]
        if i > 0:
            a[:i] = a[:i] - k[i] * a[i-1::-1]
        # Update prediction error
        e[i+1] = (1 - k[i]**2) * e[i]

    return -a
```

Feature extraction pseudo-code

```
def feature_extraction(df, window_size):
    features = []
    labels = []
    label = df.iloc[0, 3]  # Extract label from 4th column, constant per df
    for i in range(window_size, len(df), 1):  # step +win size give overlap
        win = df.iloc[i-window_size:i]
        emg1 = win.iloc[:,0].values  # Quad
        emg2 = win.iloc[:,1].values  # Calf
        strain = win.iloc[:,2].values
        rms1 = rms(emg1)
        wl1 = wl(emg1)
        ar1 = aryule(emg1, order=4)
        rms2 = rms(emg2)
        wl2 = wl(emg2)
        ar2 = aryule(emg2, order=4)
        rms_strain = rms(strain)
        feat = np.concatenate([[rms1, wl1] + list(ar1), [rms2, wl2] +
 list(ar2), [rms_strain]])
        features.append(feat)
        labels.append(label)
    return np.array(features), np.array(labels)
```

Feature concatenation pseudo-code

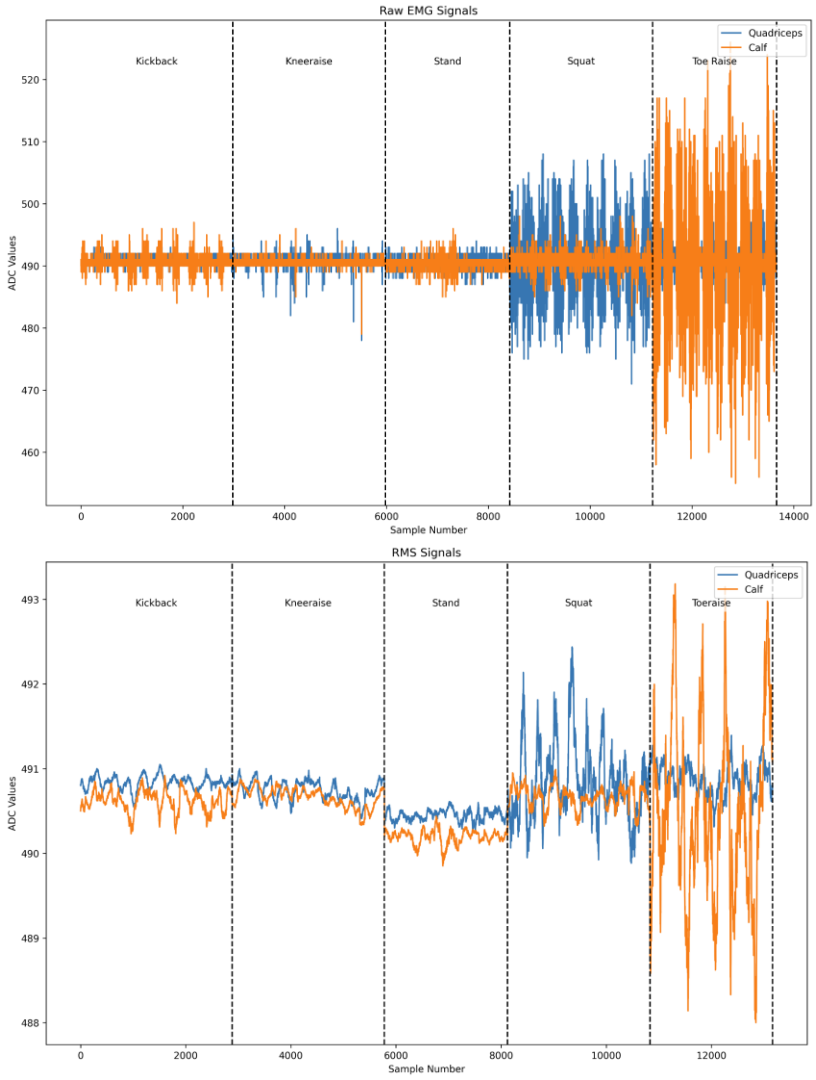
```
train_features_all = []
test_features_all = []
train_labels_all = []
test_labels_all = []
for activity_df in activity_frames:
      #activity_frames is a list of data frames, each containing one activity
      train_df, test_df = train_test_split(activity_df, train_size=0.8,
      random_state=42, shuffle=False)
```

```
            train_feat, y_train = feature_extraction(train_df, window_size=window_size)
            test_feat, y_test = feature_extraction(test_df, window_size=window_size)

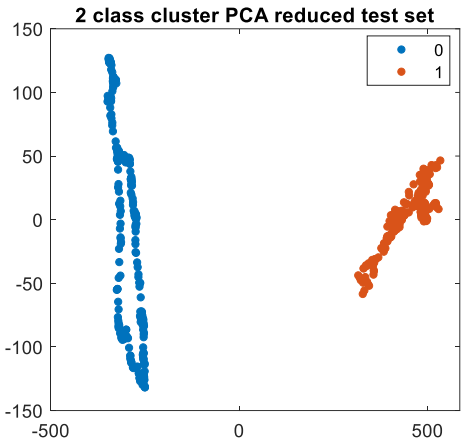
            train_features_all.append(train_feat)
            test_features_all.append(test_feat)
            train_labels_all.append(y_train)
            test_labels_all.append(y_test)

    train_features_all = np.vstack(train_features_all)
    test_features_all = np.vstack(test_features_all)
    train_labels_all = np.hstack(train_labels_all)
    test_labels_all = np.hstack(test_labels_all)
```

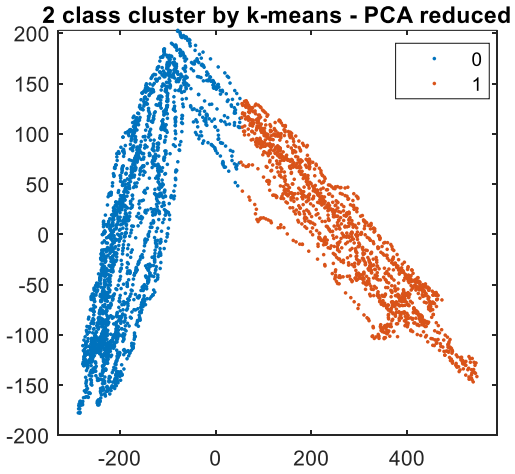
Data and feature concatenation example (Q2)

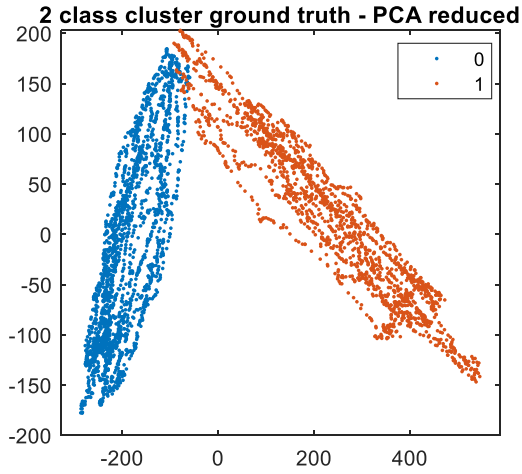


Scatterplot representation for PCA reduced data on test set



Scatterplot representation for PCA reduced data clustered using k-means





Using built in PCA

```
from sklearn.decomposition import PCA
pca_sklearn = PCA(n_components=2)
# Transform the data to 2D using the first two principal components
score2 = pca_sklearn.fit_transform(features_2cl)
```

Using built in k-means

```
from sklearn.cluster import KMeans
kmeans_sklearn = KMeans(n_clusters=2, max_iter=1000, random_state=1)
c = kmeans_sklearn.fit_predict(features_2cl)
```